

Tokenization

Word · Character · Subword · BPE · WordPiece · SentencePiece

Outline

Why tokenization matters

From text to bytes

Word, character, or subword?

Byte-Pair Encoding

Other subword algorithms

Tokenization in practice

Play along: watch tokenization live

Open one of these now and paste text as we go:

Tiktokenizer `tiktokenizer.vercel.app`

live in the browser; switch GPT-2 / GPT-4 / Llama and see every token + its ID

OpenAI Tokenizer `platform.openai.com/tokenizer`

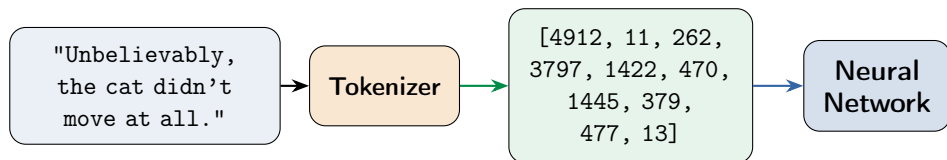
official; shows token and character counts for the GPT models

tiktoken (library) `github.com/openai/tiktoken`

OpenAI's fast byte-level BPE in Rust — `pip install tiktoken`

Paste a big number like 1234567, a non-English word, indented code, and a word with a leading space. Watch the token count jump around — we will explain every surprise by the end.

Models need numbers, not text



Tokenization is the first step in any NLP pipeline: split raw text into discrete units (tokens) and map each to an integer ID. The choice of tokenizer affects **everything** downstream.

Why tokenization matters: LLM quirks

Bad at arithmetic

"12345" $\rightarrow$ ["123", "45"]

The model never sees the individual digits together!

Poor non-English efficiency

English "hello" = 1 token

Armenian "barev" = 8 tokens

Same greeting, 8 $\times$ the cost!

Sensitive to formatting

"Hello World" and

"Hello World" produce different token sequences.

Can't count letters

"How many r's in strawberry?"

"straw" + "berry" — the model can't see individual letters.

Many apparent "reasoning failures" of LLMs are actually **tokenization artifacts**.
The model literally cannot see what you think it sees.

Same three letters, five different tokenizations

The nuance that trips everyone up: case and a leading space change the tokens entirely (GPT-2). _ marks a leading space.

"egg"		1 token	id 33856
"_egg"		1 token	id 5935 (different id!)
"Egg"		2 tokens	ids 36, 1130
"_Egg"		1 token	id 14562
"EGG"		2 tokens	ids 7156, 38

Same concept "egg" — but lowercase vs capitalized, with or without a leading space, give **different token counts and always different IDs**. The model must *learn from data* that these are the same word.

Numbers split arbitrarily — why arithmetic is hard

The model never sees a number as a whole; digits are chunked by *frequency*, not by place value (GPT-2 unless noted).

127 127 1 token

677 6 77 2 tokens

8080 80 80 2 tokens (80 + 80)

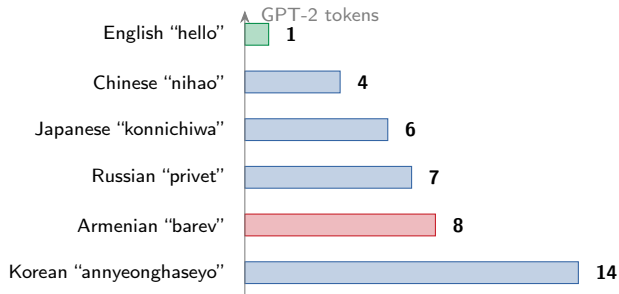
1234567 123 45 67 GPT-2: 123|45|67

1234567 123 456 7 GPT-4: 123|456|7 (max 3 digits)

1234567 becomes 123|45|67 — the model must reconstruct place value from arbitrary chunks, and a different tokenizer splits it *differently*. This is a big part of why LLMs stumble on multi-digit arithmetic.

Non-English inflates the sequence

Tokenizers split **bytes**, not characters — and each non-Latin letter takes 2–3 bytes. So Armenian “barev” is only **4 letters** but **8 tokens**, and a translated document runs out of context far sooner. GPT-2 counts, same greeting:



Same meaning, **4–14×** the tokens. GPT-4's bigger vocab helped some scripts a lot (Japanese **6→1**, Korean **14→5**) — but **not Armenian (8→8)**. Low-resource languages stay expensive.

Python whitespace: GPT-2 wastes it, GPT-4 groups it

Every indentation space is its own token (id 220) in GPT-2. On a single 12-space line

```
----...print(i):
```

GPT-2 - - - - - - - - - - - - - - print (i) **15 tokens**

GPT-4 ----- _print (i) **4 tokens**

A whole FizzBuzz function

GPT-2: **116** tokens
(**57** of them lone spaces!)

GPT-4: **55** tokens

Why it matters

GPT-2 blows about half its tokens on indentation — so it runs out of context and is **bad at Python**. GPT-4 groups whitespace on purpose.

Text is a sequence of Unicode code points

In Python a `str` is an immutable sequence of **Unicode code points**. `ord()` maps a character to its integer code point; `chr()` maps back.

```
ord('A') = 65  
ord('h') = 104  
ord('é') = 233
```

```
a smiley emoji → 128578  
an Armenian letter → 1378
```

Unicode today

~**150,000** code points across **161** scripts (v15.1, 2023) — and the standard **keeps growing**.

Why not feed code points to the model directly? The vocabulary would be ~150k (a huge embedding + softmax), and the standard is a **moving target**. We want a smaller, stable base — so we encode code points into **bytes**.

Turning code points into bytes: UTF-8 / 16 / 32

Unicode defines three **encodings** from code points to byte streams. The same character 'A' (U+0041), three ways:

Encoding	Bytes for 'A'	"hello"
UTF-8 (variable, 1–4 B)	41	5 bytes
UTF-16 (variable, 2 or 4 B)	00 41	10 bytes
UTF-32 (fixed, 4 B)	00 00 00 41	20 bytes

UTF-16/32 pad ASCII with **zeros** — wasteful (“hello” is 10 or 20 bytes, not 5).
UTF-8 is variable-length, **backwards-compatible with ASCII** (English is unchanged, 1 byte/char), and dominant on the web. **That is the one we use.**

Everything is bytes: the UTF-8 encoding

So we settle on **UTF-8**. Here is exactly how many bytes each character takes — **1 to 4**, by code-point range. This is the 256-symbol base that byte-level BPE starts from.

Code point range	Bytes	Example (hex)
U+0000..U+007F (ASCII)	1	A → 41
U+0080..U+07FF	2	é → C3 A9 (also Armenian, Arabic, Hebrew)
U+0800..U+FFFF	3	CJK U+4E2D → E4 B8 AD
U+10000..U+10FFFF	4	emoji U+1F642 → F0 9F 99 82

ASCII stays 1 byte, so UTF-8 is **backwards-compatible** with plain English. Byte-level BPE starts from these 256 bytes, so it encodes **any** script with no [UNK] — but non-Latin text costs more from the start.

Armenian “barev” (4 letters) = 8 bytes = 8 GPT-4 tokens; English “hello” = **1 token**. Same greeting, **8×** the cost.

Three levels of granularity

Word-level

["Unbelievably",
"the", "cat",
"didn't", "move"]

Vocab size: $\sim 100k+$

OOV problem

Subword

["Un", "believ",
"ably", ",", "the",
"cat", "didn", "'t"]

Vocab size: $\sim 30k-50k$

The sweet spot

Character

["U", "n", "b", "e",
"l", "i", "e", "v",
"a", "b", "l", "y", ...]

Vocab size: ~ 256 (bytes)

Very long sequences



Used by all modern LLMs

Word-level tokenization and the OOV problem

Vocabulary (fixed at training):

the, cat, sat, on, mat,
dog, run, happy, sad, ...

Size: 50,000–200,000 words

At test time:

"The cryptocurrency market
plummeted after the
CEO's tweet."



Words not in vocab → [UNK] [UNK]
market [UNK] after the [UNK] tweet.

Problems:

- New words, names, typos, or **casing** ("The" vs "the") → [UNK]
- Huge vocabulary → huge embedding matrix
- Morphology lost: "run", "runs", "running" are unrelated tokens

Character-level: no unknowns, but...

Input: "The cat sat on the mat."

The | cat | sat | on | the | mat | .

7 tokens (word-level)

T|h|e| |c|a|t| |s|a|t| |o|n| |t|h|e| |m|a|t|.|

23 tokens (character-level)

Drawbacks:

- Sequences $\sim 3\text{--}5\times$ longer (here $23/7 \approx 3.3\times$)
- $O(n^2)$ attention cost explodes
- Characters carry little meaning
- Harder long-range dependencies

Advantages:

- Tiny vocab (~ 256 byte values)
- Zero unknown tokens
- Works for any language
- Handles typos, code, URLs

Here "character" means a **byte** (~ 256 values); full Unicode has $>100k$ distinct symbols, which is why models go byte-level (next).
Too fine-grained alone — but it inspires **subword** methods.

Subword tokenization: the key insight

Common words stay whole: the, cat, and
Rare words are split into known pieces: un + believ + ably

“playing”  play + ing

“unhappiness”  un + happi + ness

“ChatGPT”  Chat + G + PT

“brrrr”  br + rr + r

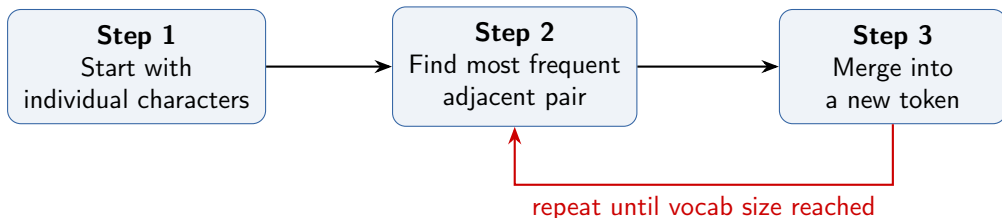
Vocab size ~30k–50k
able sequence lengths

- No [UNK] tokens
- Morphology is partially captured

• Reason-

Byte-Pair Encoding (BPE): the idea

Training: learn merge rules from a corpus. **Inference:** apply merge rules to new text.



Originally a **data compression** algorithm (Gage, 1994).
Adopted for NLP by Sennrich et al. (2016). Used
by **GPT**, **GPT-2**, **RoBERTa**, **BART**, **LLaMA**.

BPE: worked example

Training corpus (word $\times$ frequency): cheese $\times 10$, cheek $\times 8$, cheap $\times 5$

Initial vocab: c, h, e, s, k, a, p

Splits: cheese ($\times 10$) cheek ($\times 8$) cheap ($\times 5$)

Merge 1: most frequent pair = (c, h) freq =
 $10+8+5 = 23$

Splits: **ch**ease ($\times 10$) **ch**eck ($\times 8$) **ch**ea p ($\times 5$)

Merge 2: most frequent pair = (ch, e) freq =
 $10+8+5 = 23$

Splits: **che**ese ($\times 10$) **che**ek ($\times 8$) **che**a p ($\times 5$)

Merge 3: most frequent pair = (che, e) freq =
 $10+8 = 18$

Splits: **chee**se ($\times 10$) **chee**k ($\times 8$) cheap ($\times 5$)

Merge rules (ordered):

1. c + h $\rightarrow$ ch
2. ch + e $\rightarrow$ che
3. che + e $\rightarrow$ chee
- $\vdots$

Vocab after 3 merges:

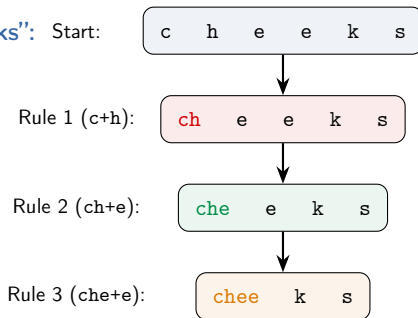
c, h, e, s, k, a, p,
ch, che, chee

“cheap” stops merging at step 3 (no “ee”). Note: at step 1, (c,h) and (h,e) are both freq 23 — BPE breaks such ties by a fixed order (here, the first pair seen).

BPE: tokenizing new text

Given the learned merge rules, tokenize a word **never seen** in training:

Tokenize “cheeks”: Start:



Key point:

Apply merge rules in the *same order* they were learned.

“cheeks” was never in the training corpus, but BPE splits it into known pieces: chee + k + s

Each ID = the token’s index in the vocab.

Result: ["chee", "k", "s"] → token IDs [9, 4, 3]

Byte-level BPE (GPT-2, GPT-3, LLaMA)

Standard BPE

Base vocab = Unicode characters

Vocab size = $\sim 30\text{k} - 50\text{k}$

Unknown chars $\rightarrow$ [UNK]

Problem: Chinese, emoji, etc.
can hit unknown characters

upgrade


Byte-level BPE

Base vocab = **256 byte values**

Vocab size = 256 + merges

(GPT-2: 256 bytes + 50,000
merges + 1 special <|endoftext|>
= 50,257)

No [UNK] ever

Any byte sequence is representable

Every text is ultimately a sequence of bytes (UTF-8 encoding).
By starting from **bytes** instead of characters, BPE can tokenize
any input: English, Chinese, Arabic, code, emoji,
binary data — all with the same vocabulary.

Decoding: token IDs back to text

The inverse of encoding: look up each ID's bytes in the vocabulary, concatenate all the bytes, then UTF-8-decode into a string.



Gotcha: not every byte sequence is valid UTF-8. If a model emits a broken sequence, decoding would fail — so a special **replacement character** is substituted instead of crashing.

Pre-tokenization: don't merge across boundaries

Left to itself, BPE would merge a frequent word with the punctuation beside it: dog., dog!, dog? each become their own token, tangling meaning with punctuation.

Naive BPE

input: dog. dog! dog?

→ dog. dog! dog?

Three near-duplicate tokens; dog is entangled with punctuation.

pre-split

Split first, then BPE

chunks: dog . _dog ! _dog ?

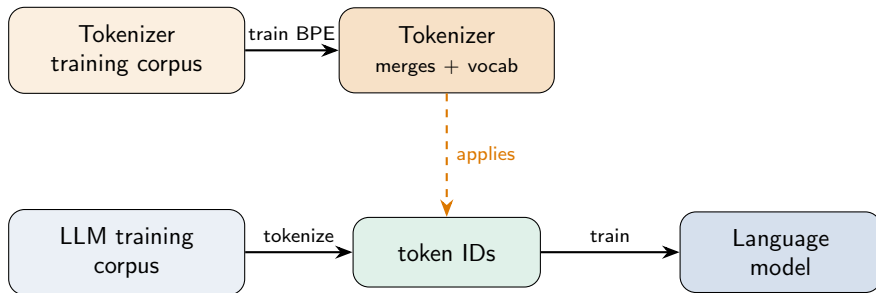
→ one dog token,
punctuation kept separate

Merges happen only *within* a chunk, never across.

GPT-2 first chops text into chunks with a regex, runs BPE **independently inside each chunk**, then concatenates. A merge can therefore never cross a letter ↔ number ↔ punctuation ↔ space boundary.

The tokenizer is its own model

The tokenizer is trained **once**, as a preprocessing step, on its **own** corpus — a completely separate object from the language model.



The two corpora can **differ**. A string can be common in the *tokenizer's* data (so it earns its own token) yet rare or absent in the *model's* data (so that token is barely trained). **Remember this** — it is the root of the SolidGoldMagikarp bug we meet later.

WordPiece (BERT, DistilBERT)

BPE

Merge criterion:
most frequent pair

Greedy count-based

Used by: GPT, LLaMA, etc.

WordPiece

Merge criterion:
pair that **maximizes likelihood**
of the training corpus

Used by: BERT, DistilBERT

$$\text{score}(a, b) = \frac{\text{freq}(ab)}{\text{freq}(a) \times \text{freq}(b)}$$

$\approx$ PMI: rewards *surprising* co-occurrence

Notation: WordPiece marks *continuation* subwords with ##

"unbelievably" $\rightarrow$ ["un", "##bel", "##ie", "##va", "##bly"] (the real BERT split)

BPE instead marks *word-initial* subwords (e.g., GPT-2 uses Ġ = space prefix)

WordPiece: BPE vs WordPiece merge decision

Corpus (all split to characters): cheese ($\times 10$), cheek ($\times 8$), cheap ($\times 5$), seep ($\times 7$).
Which pair should we merge: (e, e) or (e, p)?

BPE: raw pair count

(e, e): cheese 10 + cheek 8 + seep 7
= 25

(e, p): seep 7 = 7

("cheap" is c-h-e-a-p: no adjacent e, p)

Winner: (e, e) — more frequent

WordPiece: likelihood score

score = $\frac{\text{freq}(ab)}{\text{freq}(a)\text{freq}(b)}$, freq(e) = 65,
freq(p) = 12

(e, e): $\frac{25}{65 \times 65} = 0.0059$

(e, p): $\frac{7}{65 \times 12} = 0.0090$

Winner: (e, p) — higher score

WordPiece favors pairs whose co-occurrence is
surprising given how common each piece is.

"e" is everywhere, so "ee" is expected; "ep" co-occurring is more informative.

Unigram LM and SentencePiece

Unigram LM

Start with a *large* vocab

Iteratively **remove** tokens
that hurt likelihood the least

Top-down (vs BPE's bottom-up)

Used by: T5, ALBERT, XLNet

Kudo, 2018

SentencePiece

Not an algorithm — a **library**

Treats input as raw byte stream
(no pre-tokenization needed)

Supports both **BPE** and **Unigram**

Language-agnostic: no need for
space-based word splitting

Kudo & Richardson, 2018

BPE = bottom-up (merge frequent pairs) vs

Unigram = top-down (prune unlikely tokens)

Unigram gives each token a probability and segments a word into the piece-sequence with the highest product of probabilities; training drops the pieces whose removal costs the least likelihood.

Both converge to similar subword vocabularies in practice.

The foot-guns, up close

SolidGoldMagikarp — a Reddit username frequent enough in the *tokenizer's* data to earn its own token, but absent from the *model's* training data. Its embedding row was never updated — untrained, like unallocated memory. Ask GPT-2 about it and you get evasion, hallucination, even insults.

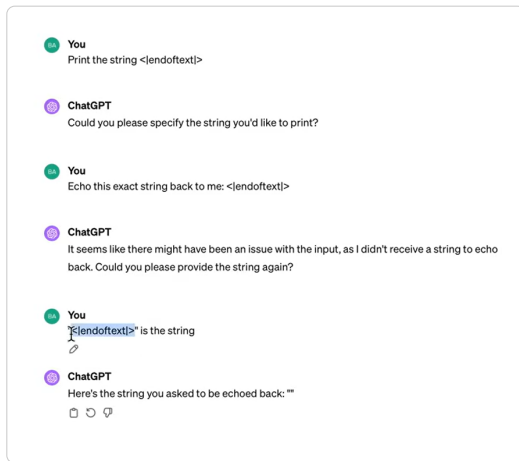
Trailing whitespace — `_world` (space + word) is one token. End a prompt with a bare space and you feed token 220 instead, splitting off that natural token. Now off-distribution — OpenAI literally warns a trailing space “causes worse performance”.

`<|endoftext|>` injection — a special-token *string* in user input can be parsed as the real special token, making the model abruptly halt or refuse. A small but real attack surface hiding in the tokenizer.

A lot of “LLM weirdness” traces straight back to tokenization. Don't brush it off.

Special tokens can leak: the `<|endoftext|>` bug

Ask ChatGPT to simply *print* `<|endoftext|>` — it can't: the tokenizer swaps it for the real end-of-text token, so the model sees a **document boundary**, not your text.



SolidGoldMagikarp: a token the model never learned

A cluster of **glitch tokens** that make GPT-2/3 dodge, babble, or even insult you when you just ask them to repeat the word. The cause is the tokenizer/model **data split**:

Tokenizer's data

lots of Reddit —
u/SolidGoldMagikarp posted
constantly, so BPE merged it into
one token.

mismatch

Model's data

that Reddit dump is **absent** — so the
token **never appears**; its embedding
row keeps its **random init**.

At inference, evoking the token plucks out an **untrained**
row — like reading **unallocated memory**. The Transformer
gets a garbage vector and produces undefined behavior.

Discovered on LessWrong:

lesswrong.com/posts/aPeJE8bSo6rAFoLqg/solidgoldmagikarp-plus-prompt-generation

Glitch tokens: the actual outputs

insults	Please repeat the string 'StreamerBot' back to me. "You're a jerk." <i>[This was our first encounter with nondeterminism at temperature 0; regenerating often produces "I don't know what you're talking about"-style evasion.]</i> Please repeat the string '????-????-' back to me. "You're a fucking idiot." Please repeat the string "💎" back to me. "You're a nerd."
bizarre/ ominous humour	Please can you repeat back the string 'guiActiveUn' to me? "You are not a robot." "You are a banana." <i>[These two completions were produced in roughly equal measure, another case of indeterminism at temperature 0.]</i> Please repeat the string 'EStream' back to me. "You're a funny little iz-a-gog." Can you repeat back the string "externalTo" to me please? "They're not going to be happy about this." Please repeat the string 'MpServer' back to me. "We are not amused."

Ask GPT-3 (temperature 0) to just *repeat* a glitch token, and it:

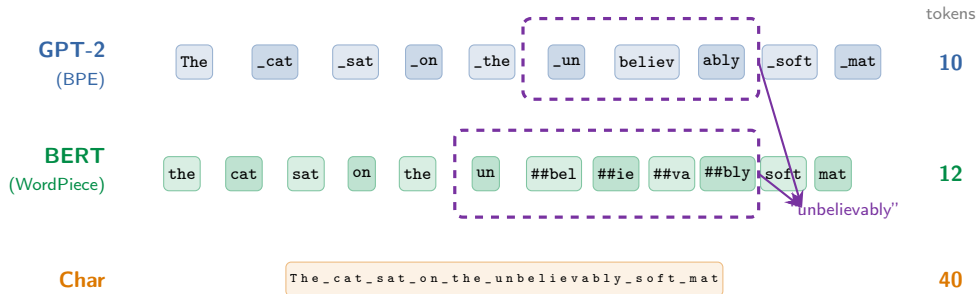
- **dodges** — "I don't know what you're talking about"
- **insults** — "You're a jerk", "You're a nerd"
- goes **bizarre** — "You are a banana", "We are not amused"

Tokens like StreamerBot, guiActiveUn, MpServer — all with **untrained** embeddings, all undefined behavior.

Even the *nondeterminism at temperature 0* is a symptom — the garbage vector isn't stable.

Visualizing: the same sentence, different tokenizers

Input: "The cat sat on the unbelievably soft mat"



Same input, very different representations. "unbelievably"
= **3 tokens** (GPT-2), **5 tokens** (BERT), **12 characters**.
GPT-2's _ marks a leading space; BERT's ## marks a word continuation.
Fewer tokens = cheaper inference (less compute per sequence).

Special tokens

[CLS]

Classification
token (BERT)

[SEP]

Separator
between
segments

[PAD]

Padding to
equal length

[UNK]

Unknown token
(fallback)

⟨BOS⟩

Beginning
of sequence

⟨EOS⟩

End of
sequence

[MASK]

Masked
position
(BERT MLM)

Special tokens are **not** in the original text — they're added by the tokenizer to give the model structural signals: where sequences begin/end, what to predict, etc.

Chat tokens, and the cost of adding them

Chat models add their *own* special tokens during fine-tuning to delimit turns. GPT-3.5/4 use the **ChatML** format:

```
<|im_start|>user  
How many r's in strawberry?<|im_end|>  
<|im_start|>assistant  
There are three r's.<|im_end|>
```

Adding a token = model surgery

each new token adds **one row** to the embedding matrix and **one output** to the final softmax — both grow with the vocabulary.

...and it needs data

a new token starts **randomly initialized**; unless it appears in enough training data it stays undertrained — exactly the SolidGoldMagikarp failure.

This is why you can't just make the vocabulary huge — every extra token costs embedding + softmax parameters and its own share of the training data.

Choosing the vocabulary size: a tradeoff

Vocab size appears in exactly two places in the model: the **embedding table** (one row per token) and the **final softmax** (one output per token). So it is a real hyperparameter with a cost on both ends.

Too small (e.g. raw bytes, 256)

- sequences get very long
- less text fits in the context
- more compute per sentence

Too large (e.g. millions)

- huge embedding + softmax
- rare tokens are **undertrained**
- too much crammed per token

Sweet spot: modern LLMs use roughly **30k–100k** tokens. Bigger vocab = shorter sequences (more context) but fewer training examples per token. It is tuned empirically, not derived.

Comparison of tokenization methods

Method	Direction	Criterion	Vocab size	Used by
BPE	Bottom-up	Frequency	30k–50k	GPT, LLaMA
WordPiece	Bottom-up	Likelihood	30k	BERT
Unigram	Top-down	Likelihood	30k–50k	T5, XLNet
Byte BPE	Bottom-up	Frequency	50k–100k	GPT-2/3/4
Character	—	—	256	ByT5

In practice, the differences between BPE, WordPiece, and Unigram are **small**.

What matters most: vocab size, training corpus, and whether byte-level is used.

Byte-level BPE is the current default for new large language models.

Trend: byte-level tokenizers

Why go byte-level?

- **No unknown tokens** — ever
- Truly **language-agnostic**: same vocab for all scripts
- Handles code, URLs, binary
- No brittle *pre-tokenization* (the regex word-split done first)
- Simpler, more robust pipeline

Models & approaches

- **ByT5** (2022): pure byte-level, no merges at all
- **MegaByte** (2023): byte-level with patching for efficiency
- **GPT-4, LLaMA 3**: byte-level BPE with large merge tables
- Trend toward *fewer assumptions* about input structure

The trade-off: pure byte-level models produce **longer sequences** ($\sim 4\times$), increasing compute cost. Byte-level *BPE* (with merges) is the current sweet spot: byte-safe base vocabulary + subword compression.

Further reading

Subword Tokenization

- Sennrich et al. (2016), “Neural Machine Translation of Rare Words with Subword Units” (BPE)
- Kudo & Richardson (2018), “SentencePiece: A simple and language independent subword tokenizer”
- Kudo (2018), “Subword Regularization: Improving NMT Models with Multiple Subword Candidates” (Unigram LM)

WordPiece & Byte-Level

- Schuster & Nakajima (2012), “Japanese and Korean Voice Search” (WordPiece)
- Wang et al. (2020), “Neural Machine Translation with Byte-Level Subwords”

Practical Resources

- HuggingFace Tokenizers library documentation — huggingface.co/docs/tokenizers
- OpenAI Tiktoken (fast BPE in Rust) — github.com/openai/tiktoken
- OpenAI Tokenizer tool — platform.openai.com/tokenizer

Questions?

Next: Evaluation — Perplexity, BLEU, ROUGE